Replace and modernize legacy VPN architectures with a cloud-native Zscaler Zero Trust solution to reduce attack surface and lateral movement.

TITLE: Replace and modernize legacy VPN architectures with a cloud-native Zscaler Zero Trust solution to reduce attack surface and lateral movement.

COLLEGE NAME: KPR COLLEGE OF ARTS SCIENCE AND RESEARCH

DEPARTMENT: B.Sc. COMPUTER SCIENCE

LIST OF TEAMMATES :

S.NO	NAME	COLLEGE	CONTACT
01	PAVITHRA V	KPR College of Arts Science and Research	23bcs043@kprcas.ac.in
02	PRANESH J	KPR College of Arts Science and Research	23bcs044@kprcas.ac.in
02	PREETHIKA P	KPR College of Arts Science and Research	23bcs047@kprcas.ac.in
04	KIRUBHAKARAN V	KPR College of Arts Science and Research	23bcs035@kprcas.ac.in

INTRODUCTION :

This project outlines a strategic shift in our organization's security architecture: the complete replacement of our legacy VPN infrastructure with a cloud-native Zero Trust solution from Zscaler. The primary objective is to drastically reduce our attack surface and eliminate the risk of lateral movement, two critical vulnerabilities inherent in traditional network security models.

Our current VPN architecture operates on an outdated "castle-and-moat" model. By granting authenticated users broad, network-level access, it places remote devices directly onto the internal network. This creates a flat network where a single compromised credential can allow attackers to move laterally, scan for vulnerable systems, and exfiltrate sensitive data. Additionally, reliance on on-premises hardware exposes public-facing IP addresses and ports, making them constant targets for automated attacks. These factors, combined with the operational overhead of managing physical infrastructure and poor user experience from traffic backhauling, render the legacy model both insecure and inefficient.

To address these deficiencies, this project will implement Zscaler Private Access (ZPA). This cloud-native platform enforces a "never trust, always verify" model, shifting access control from a network-centric to an application-centric approach. By making internal applications invisible to the internet and establishing direct, encrypted connections between verified users and specific applications, Zscaler ensures users connect only to what they are authorized to use—not the underlying network. This architecture renders the internal network invisible and neutralizes any possibility of lateral movement.

The transition to Zscaler will deliver transformative outcomes. It will fortify our cyber defenses by eliminating the VPN attack surface and containing potential breaches, while also enhancing agility and user experience. The cloud-delivered model simplifies operations, removes the burden of managing legacy hardware, and optimizes performance for our hybrid workforce. Ultimately, this initiative future-proofs our organization, creating a secure, high-performance access framework that supports evolving business needs.

ABSTRACT :

Legacy VPN infrastructures were designed for a different era—when corporate networks had clear perimeters and all users worked from within office boundaries. Today, with hybrid workforces and cloud adoption, these traditional models have become security liabilities. This project proposes their complete replacement with a cloud-native Zscaler Zero Trust architecture to address these modern challenges.

The current VPN model creates two fundamental problems. First, publicly exposed VPN concentrators with open ports invite constant scanning and attacks. Second, once authenticated, users gain network-level access that allows threats to move laterally across the network. A single compromised credential can potentially lead to widespread breach. Additionally, traffic backhauling through data centers degrades user experience, while hardware appliance maintenance creates operational overhead.

The proposed Zscaler solution fundamentally rearchitects remote access. By making internal applications invisible to the internet and establishing direct user-to-application connections, the platform eliminates the attack surface entirely. Users verify their identity and device posture before gaining access—and that access is limited strictly to specific applications, not the underlying network. This micro-tunneling approach renders lateral movement impossible because attackers cannot see or reach other resources.

Beyond security, this transformation delivers operational benefits. The cloud-native model eliminates hardware management, scales automatically, and routes traffic efficiently through Zscaler's global edge network for optimal performance. By replacing an outdated castle-and-moat model with modern Zero Trust principles, this project creates a security framework ready for the challenges of tomorrow.

USE CASE SCENARIOS:

Scenario 1: Emergency Remote Access During Disaster Recovery

A data center outage occurs due to a regional power failure. The disaster recovery team needs emergency access to backup systems hosted at a secondary site. ZPA provides immediate, policy-based access without exposing recovery environments to the internet.

Output:

- User identity verified (DR_Team_Group)
- Emergency access protocol triggered (auto-escalation approved)
- Multi-factor authentication enforced (hardware token required)
- Session limited to recovery systems only with full audit logging • Automatic timeout after 4 hours of inactivity

Scenario 2: Compliance Auditor Reviewing Financial Records

An internal compliance auditor needs to review quarterly financial transactions stored across multiple departmental servers. ZPA enables cross-application access while maintaining strict segregation between financial systems.

Output:

- User identity verified (Internal_Auditor_Group)
- Cross-application access granted (Finance ERP + Document Storage)
- Read-only audit mode enforced (no modifications permitted)
- All sessions recorded for regulatory compliance (SOX/PCI)
- Audit trail generated with timestamps and accessed records

Scenario 3: Contractor Onboarding for Specific Project

A UI/UX contractor is hired for a 3-month website redesign project. They need access to development servers and design repositories but must be isolated from production environments and internal communications.

Output:

- User identity verified (External_Contractor_Group)

- Project-based access provisioned (90-day validity)
- Environment restriction enforced (Dev/Staging only – no Production)
- Design repository access granted (Figma/GitLab integration)
- Automatic access revocation on contract end date

Scenario 4: Researcher Sharing Data with Collaborative Institution

A research team needs to temporarily share a sensitive dataset with a partner institution for joint analysis. The dataset must remain on-premises while providing secure, time-limited access to external collaborators.

Output:

- External collaborator identities verified (Partner_University_Group)
- Dataset-specific access granted (time-limited 30 days)
- Download restrictions enforced (view-only with watermarking)
- Access logs shared with both institutions' security teams
- Automatic expiration with no renewal option

Scenario 5: Executive Access During International Travel

A university dean traveling internationally needs access to confidential board meeting documents and donor records. ZPA provides secure access while enforcing additional security controls due to high-risk travel location.

Output:

- User identity verified (Executive_Leadership_Group)
- Geo-location risk assessment triggered (travel country flagged)

- Step-up authentication enforced (biometric + MFA)
- Session duration limited (2-hour maximum per session)
- Real-time threat monitoring activated for duration of travel

Scenario 6: Intern Access to Learning Resources and Sandbox Environment

Summer interns require access to training materials, documentation, and isolated sandbox environments to learn internal tools without risking exposure to production systems or sensitive data.

Output:

- User identity verified (Intern_Group)
- Sandbox environment access granted (isolated from production)
- Training repository access provided (read-only for materials)
- Time-based restriction enforced (9:00 AM–6:00 PM weekdays)
- Automatic access deactivation at internship conclusion

STAGE – 1

OVERVIEW :-

This project focuses on replacing and modernizing traditional Virtual Private Network (VPN) architectures with a cloud-native Zero Trust security solution using Zscaler. In the current digital environment, organizations are increasingly dependent on remote access to internal applications and systems. Traditional VPN technology was designed for an earlier era when most users worked within office networks and applications were hosted inside corporate data centers. However, with the rise of cloud computing, hybrid work environments, and

increasing cyber threats, legacy VPN architectures are no longer sufficient to provide strong security and efficient access control.

Traditional VPN models operate on a “connect first, verify later” approach. Once a user successfully authenticates through the VPN gateway, they are granted broad network access and become part of the internal network. This architecture follows the “castle-and-moat” security model, where the internal network is trusted and the external internet is considered untrusted. While this model worked in the past, it introduces major security risks today. If an attacker compromises a user’s credentials or device, they can gain access to the internal network and move laterally across systems, potentially reaching critical assets such as databases, research repositories, or financial applications.

Another major limitation of legacy VPN solutions is the increased attack surface. VPN concentrators must expose ports to the public internet to allow remote connections, making them attractive targets for attackers who attempt brute-force login attempts, vulnerability exploitation, or port scanning. Additionally, VPN architectures often cause performance issues due to “traffic hair-pinning,” where user traffic is routed through a central data center before reaching its destination. This leads to latency, slower application performance, and poor user experience, especially for distributed or remote workforces.

To address these limitations, this project proposes implementing a Zero Trust Network Access (ZTNA) architecture using the Zscaler Zero Trust platform. Zero Trust is a modern security framework built on the principle of “never trust, always verify.” Instead of trusting users based on their network location, Zero Trust continuously verifies identity, device security posture, and contextual factors before granting access to applications. This approach ensures that users only receive the minimum level of access required to perform their tasks, reducing the risk of unauthorized access and security breaches.

The core component of the solution is Zscaler Private Access (ZPA), a cloud-based service that connects users directly to specific applications rather than to the entire network. When a user requests access, ZPA authenticates their identity through integration with identity

providers such as Azure Active Directory and performs security checks on the device. After successful verification, the system establishes a secure encrypted micro-tunnel that connects the user only to the authorized application. This method prevents attackers from discovering or accessing other systems in the network because the internal infrastructure remains invisible to external users.

One of the most significant benefits of this architecture is the reduction of the organization's attack surface. Since internal applications are not exposed to the internet and no inbound firewall ports are required, attackers cannot scan or directly target these systems. Furthermore, the micro-segmentation capability of Zero Trust prevents lateral movement, meaning that even if a user account is compromised, the attacker cannot move to other resources within the network.

In addition to improved security, the Zscaler Zero Trust solution enhances operational efficiency and user experience. Because the platform is cloud-native, it eliminates the need for maintaining on-premises VPN hardware and simplifies network management. Users can access applications securely from any location without performance issues caused by centralized network routing. This approach also supports the modern hybrid workforce by providing secure, scalable, and reliable access to organizational resources.

Overall, this project aims to build a modern and secure network architecture. By replacing legacy VPN infrastructure with a cloud-based Zero Trust solution using Zscaler, the organization can reduce cyber risks, prevent unauthorized access, and provide secure application access for users from any location.

LIST OF VULNERABILITY TABLE :

S.No	Category Name	Vulnerability Name	CWE/CVE - No

01.	**Broken Access Control**	**Relative Path Traversal**	**CWE-23**
02.	**Security Misconfiguration**	**J2EE Misconfiguration: Data Transmission Without Encryption**	**CWE-5**
03.	**Software Supply Chain Failures**	**Unimplemented or Unsupported Feature in UI**	**CWE-447**
04.	**Cryptographic Failures**	**Weak Encoding for Password**	**CWE-261**
05.	**Injection**	**Improper Input Validation**	**CWE-20**
06.	**Insecure Design**	**External Control of File Name or Path**	**CWE-73**
07.	**Authentication Failures**	**Empty Password in Configuration File**	**CWE-258**
08.	**Software or Data Integrity Failures**	**Insufficient Verification of Data Authenticity**	**CWE-345**
09.	**Security Logging & Alerting Failures**	**Improper Output Neutralization for Logs**	**CWE-117**

10.	**Mishandling of Exceptional Conditions**	**Generation of Error Message Containing Sensitive Information**	**CWE-209**

REPORT:-

1.

Vulnerability Name:- Relative Path Traversal

CWE/CVE : - CWE-23

OWASP/SANS Category:- A01:2025 Broken Access Control

Description:- The application uses externally supplied input to construct file or directory paths. However, it does not properly validate or sanitize path traversal sequences such as ../ or ..\. An attacker can exploit this weakness to access files and directories outside the intended restricted directory. This may allow unauthorized access to sensitive files on the server.

Business Impact::- If exploited, this vulnerability could allow attackers to access sensitive files stored on the server that are outside the intended directory. These files may include:

- Application configuration files
- System files
- Source code
- User data
- Authentication credentials
- Database connection details

Unauthorized access to such files can lead to information disclosure, privilege escalation, and further system compromise. In several cases attackers may retrieve configuration files containing credentials that could allow them to access backend systems or databases, potentially leading to data breaches, service disruption, or complete application compromise. This vulnerability may also expose the organization to regulatory penalties, reputational damage, and loss of customer trust if sensitive or personal data is accessed.

```c
# include <stdio.h>
# include <string.h>
# include <wchar.h>
# include <sys/types.h>
# include <sys/stat.h>
# include <fcntl.h>
# include <unistd.h>
# include <stdlib.h>
# define BASEPATH "/tmp/"
# define FILENAME_MAX 512

static void Relative_Path_Traversal(void)
{
    char * data;
    char data_buf[FILENAME_MAX] = BASEPATH;
    char sub_buf[FILENAME_MAX];

    if (fgets(sub_buf, FILENAME_MAX, stdin) == NULL) exit (1);
    data = data_buf;
    strcat(data, sub_buf);

    FILE *file = NULL;
    file = fopen(data, "wb+");  //Noncompliant
    if (file != NULL) fclose(file);
}

int path_call(void){
    Relative_Path_Traversal();
}
```

2.

Vulnerability Name:- J2EE Misconfiguration: Data Transmission Without Encryption

CWE/CVE : - CWE-5

OWASP/SANS Category:- A02:2025 Security Misconfiguration

Description:- Information transmitted over the network is not properly encrypted. When sensitive data is sent in plaintext or using weak encryption mechanisms, attackers who intercept network traffic may be able to read, capture, or modify the transmitted information. This vulnerability commonly occurs when applications use unsecured protocols such as HTTP instead of HTTPS or when encryption mechanisms are improperly configured.

Business Impact::- If sensitive information is transmitted without proper encryption, attackers may intercept network traffic and gain access to confidential data such as login credentials, session tokens, personal information, or other sensitive business data. This could lead to unauthorized account access, session hijacking, data theft, or manipulation of transmitted information. In addition, exposure of sensitive data may result in regulatory compliance violations, financial losses, reputational damage, and loss of customer trust for the organization.

Example - 1

The following code attempts to establish a connection to a site to communicate sensitive information.

```
URL u = new URL("http://www.secret.example.org/");HttpURLConnection hu = (HttpURLConnection) u.openConnection();hu.set
// //...//
try {}catch (IOException e) {}
```

Though a connection is successfully made, the connection is unencrypted and it is possible that all sensitive data sent to or received from the server will be read by unintended actors.

Example - 2

In 2022, the OT:ICEFALL study examined products by 10 different Operational Technology (OT) vendors. The researchers reported 56 vulnerabilities and said that the products were "insecure by design" [REF-1283]. If exploited, these vulnerabilities often allowed adversaries to change how the products operated, ranging from denial of service to changing the code that the products executed. Since these products were often used in industries such as power, electrical, water, and others, there could even be safety implications.

Multiple vendors used cleartext transmission of sensitive information in their OT products.

3.

Vulnerability Name:- Unimplemented or Unsupported Feature in UI

CWE/CVE : - CWE-447

OWASP/SANS Category:- A03:2025 Software Supply Chain Failures

Description:- The application user interface presents a security feature that appears to be functional and provides feedback suggesting that it is active. However, the underlying functionality of this feature is not implemented or supported in the backend. As a result, users may believe that a security control is protecting the system when in reality it is not enforced.

Business Impact::- If a security feature is displayed in the user interface but not actually implemented, users and administrators may rely on it for protection while the system remains unprotected. This creates a false sense of security and may allow attackers to bypass expected security controls. Such weaknesses may lead to unauthorized access, data exposure, or misuse of application functionality. Additionally, relying on non-functional security controls can

increase the risk of security breaches, regulatory non-compliance, reputational damage, and potential financial losses for the organization.

Selected Observed Examples

Note: this is a curated list of examples for users to understand the variety of ways in which this weakness can be introduced. It is not a complete list of all CVEs that are related to this CWE entry.

Reference	Description
CVE-2000-0127	GUI configuration tool does not enable a security option when a checkbox is selected, although that option is honored when manually set in the configuration file.
CVE-2001-0863	Router does not implement a specific keyword when it is used in an ACL, allowing filter bypass.
CVE-2001-0865	Router does not implement a specific keyword when it is used in an ACL, allowing filter bypass.
CVE-2004-0979	Web browser does not properly modify security setting when the user sets it.

Summary

An Unimplemented or Unsupported Feature in the UI vulnerability in Juniper Networks Junos OS on QFX5000 Series and EX4600 Series. When certain unsupported match conditions (ip-source-address, ip-destination-address, or arp-type) are used in an ethernet switching filter and applied as an output filter, the configuration can be committed but the filter will not actually take effect, causing it to bypass the intended security policy.

Impact

An unauthenticated, network-based attacker can cause a minor integrity impact to downstream networks. The vulnerability allows traffic that should be filtered by ethernet switching output filters to pass through unfiltered when these unsupported match conditions are configured. This could result in unwanted traffic reaching downstream network segments that the filter was intended to protect.

4.

Vulnerability Name:- Weak Encoding for Password

CWE/CVE : - CWE-261

OWASP/SANS Category:- A04:2025 Cryptographic Failures

Description:- Obscuring a password with a trivial encoding does not protect the password. Password management issues occur when a password is stored in plaintext in an application's properties or configuration file. A programmer can attempt to remedy the password management problem by obscuring the password with an encoding function, such as base 64 encoding, but this effort does not adequately protect the password.

Business Impact::- If passwords are stored or transmitted using weak encoding mechanisms such as Base64 or other reversible encoding methods, attackers may easily decode the encoded values and retrieve the original passwords. This can lead to unauthorized access to user accounts, administrative systems, or backend services that rely on those credentials. Once the credentials are exposed, attackers may perform further malicious activities such as privilege escalation, data theft, modification of sensitive information, or complete system compromise.

Additionally, exposure of user credentials may result in account takeovers, loss of confidential data, regulatory compliance violations, reputational damage, and financial losses for the organization.

Example 1

The following code reads a password from a properties file and uses the password to connect to a database.

Example Language: **Java** (bad code)

```
...
Properties prop = new Properties();
prop.load(new FileInputStream("config.properties"));
String password = Base64.decode(prop.getProperty("password"));
DriverManager.getConnection(url, usr, password);
...
```

This code will run successfully, but anyone with access to config.properties can read the value of password and easily determine that the value has been base 64 encoded. If a devious employee has access to this information, they can use it to break into the system.

5.

Vulnerability Name:- Improper Input Validation

CWE/CVE : - CWE-20

OWASP/SANS Category:- A05:2025 Injection

Description:- The application receives input or data from users or external sources but fails to properly validate or sanitize the input before processing it. Without proper validation, malicious or unexpected input may be processed by the application, potentially leading to security vulnerabilities such as injection attacks, system crashes, or unauthorized access.

Business Impact::- Improper input validation can allow attackers to submit malicious data that may manipulate application behavior. This could result in injection attacks, unauthorized access, data manipulation, or disclosure of sensitive information. If successfully exploited, attackers may gain access to backend systems, modify application data, or disrupt application functionality. This could lead to data breaches, service disruption, reputational damage, regulatory compliance issues, and financial losses for the organization.

Example 1

This example demonstrates a shopping interaction in which the user is free to specify the quantity of items to be purchased and a total is calculated.

Example Language: **Java** (bad code)

```
...
public static final double price = 20.00;
int quantity = currentUser.getAttribute("quantity");
double total = price * quantity;
chargeUser(total);
...
```

The user has no control over the price variable, however the code does not prevent a negative value from being specified for quantity. If an attacker were to provide a negative value, then the user would have their account credited instead of debited.

6.

Vulnerability Name:- External Control of File Name or Path

CWE/CVE : - CWE-73

OWASP/SANS Category:- A06:2025 Insecure Design

Description:- This could allow an attacker to access or modify system files or other files that are critical to the application. Path manipulation errors occur when the following two conditions are met:

1. An attacker can specify a path used in an operation on the filesystem.
2. By specifying the resource, the attacker gains a capability that would not otherwise be permitted.

Business Impact::- If exploited, this vulnerability could allow attackers to access, modify, or overwrite sensitive files on the server. These files may include system configuration files, application source code, or data files that are critical to the application. Unauthorized file access may lead to data leakage, modification of important system files, or disruption of application functionality. In severe cases, attackers may gain deeper access to the system environment, potentially leading to privilege escalation, system compromise, reputational damage, and financial loss to the organization.

Example 1

The following code uses input from an HTTP request to create a file name. The programmer has not considered the possibility that an attacker could provide a file name such as "../../tomcat/conf/server.xml", which causes the application to delete one of its own configuration files (CWE-22).

Example Language: **Java** *(bad code)*

```
String rName = request.getParameter("reportName");
File rFile = new File("/usr/local/apfr/reports/" + rName);
...
rFile.delete();
```

7.

Vulnerability Name:- Empty Password in Configuration File

CWE/CVE : - CWE-258

OWASP/SANS Category:- A07:2025 Authentication Failures

Description:- The application or system configuration file contains an account with an empty password value. Using an empty string as a password weakens the authentication mechanism and may allow unauthorized users to access the system or associated services without proper credentials. Storing empty passwords in configuration files exposes critical resources and increases the risk of unauthorized access to application components, databases, or backend systems.

Business Impact::- If a configuration file contains accounts with empty passwords, attackers may be able to gain unauthorized access to application components, services, or databases without authentication. This could allow malicious users to manipulate application data, retrieve sensitive information, or disrupt system operations. Successful exploitation may lead to data breaches, unauthorized system access, service disruption, and compromise of critical business information. Additionally, such weaknesses may result in regulatory compliance issues, reputational damage, and potential financial losses for the organization.

Example 1

The following examples show a portion of properties and configuration files for Java and ASP.NET applications. The files include username and password information but the password is provided as an empty string.

This Java example shows a properties file with an empty password string.

Example Language: **Java** (bad code)

```
# Java Web App ResourceBundle properties file
...
webapp.ldap.username=secretUsername
webapp.ldap.password=
...
```

The following example shows a portion of a configuration file for an ASP.Net application. This configuration file includes username and password information for a connection to a database and the password is provided as an empty string.

Example Language: **ASP.NET** (bad code)

```
...
<connectionStrings>
<add name="ud_DEV" connectionString="connectDB=uDB; uid=db2admin; pwd=; dbalias=uDB;" providerName="System.Data.Odbc" />
</connectionStrings>
...
```

An empty string should never be used as a password as this can allow unauthorized access to the application. Username and password information should not be included in a configuration file or a properties file in clear text. If possible, encrypt this information and avoid CWE-260 and CWE-13.

8.

Vulnerability Name:- Insufficient Verification of Data Authenticity

CWE/CVE : - CWE-345

OWASP/SANS Category:- A08:2025 Software or Data Integrity Failures

Description:- The product does not sufficiently verify the origin or authenticity of data, in a way that causes it to accept invalid data.

Business Impact::- If the application does not properly verify the authenticity or integrity of incoming data, attackers may be able to inject or manipulate malicious data that the system will accept as legitimate. This could allow unauthorized modification of application data or execution of unintended operations.Exploitation of this vulnerability may lead to data tampering, unauthorized transactions, or manipulation of system processes. In severe cases, attackers could compromise application integrity, disrupt business operations, or gain unauthorized access to sensitive information. Such incidents may result in loss of data integrity, reputational damage, regulatory compliance violations, and potential financial losses for the organization.

Example - 1

In 2022, the OT:ICEFALL study examined products by 10 different Operational Technology (OT) vendors. The researchers reported 56 vulnerabilities and said that the products were "insecure by design" [REF-1283]. If exploited, these vulnerabilities often allowed adversaries to change how the products operated, ranging from denial of service to changing the code that the products executed. Since these products were often used in industries such as power, electrical, water, and others, there could even be safety implications.

Multiple vendors did not sign firmware images.

9.

Vulnerability Name:- Improper Output Neutralization for Logs

CWE/CVE : - CWE-117

OWASP/SANS Category:- A09:2025 Security Logging & Alerting Failures

Description:- The product constructs a log message from external input, but it does not neutralize or incorrectly neutralizes special elements when the message is written to a log file.

Business Impact::- If log messages include unvalidated or unsanitized user input, attackers may inject malicious or misleading content into log files. This could allow attackers to manipulate log records, hide malicious activities, or insert fake entries that mislead administrators during security investigations. Such manipulation can compromise the reliability and integrity of logging systems, making it difficult to detect security incidents or trace attacker activities. In severe cases, attackers may exploit this weakness to cover their tracks, evade detection, or disrupt monitoring and auditing processes. This may lead to delayed incident response, security monitoring failures, compliance violations, reputational damage, and potential financial impact for the organization.

Python:

In Python, a solid mitigation strategy would be using the built-in `encode()` function on all dynamic parts before inserting them into a log message. An example snippet could look like this:

```
username_sanitized = username.encode()
logger.info(f"User {username_sanitized} logged in.")
```

Another strategy would be to use the `logging-formatter-anticrlf` logging library which can be applied on a logging handler to automatically encode CRLF characters.

JavaScript:

We are not aware of a community-tested function that handles CRLF characters securely in JavaScript. A strategy could be to manually replace all CRLF characters whilst preserving the intent for auditing. A sample snippet using a regex would look like this:

```
input.replace(/[\n]/g, '\\n').replace(/[\r]/g, '\\r');
```

10.

Vulnerability Name:- Generation of Error Message Containing Sensitive Information

CWE/CVE : - CWE-209

OWASP/SANS Category:- A10:2025 Mishandling of Exceptional Conditions

Description:- The product generates an error message that includes sensitive information about its environment, users, or associated data.

Business Impact::- If sensitive information is exposed through error messages, attackers may gain insights into the application's internal structure, configuration settings, database queries, or system paths. This information can help attackers identify weaknesses in the system and plan further attacks. Exposure of such details may lead to information disclosure, increased risk of exploitation, and unauthorized access to sensitive resources. Additionally, attackers may leverage the revealed information to conduct more sophisticated attacks such as injection, privilege escalation, or system compromise. This could result in data breaches, reputational damage, regulatory compliance issues, and financial losses for the organization.

Example 1

In the following example, sensitive information might be printed depending on the exception that occurs.

Example Language: **Java** (bad code)

```
try {
  /.../
}
catch (Exception e) {
  System.out.println(e);
}
```

If an exception related to SQL is handled by the catch, then the output might contain sensitive information such as SQL query structure or private information. If this output is redirected to a web user, this may represent a security problem.

This code tries to open a database connection, and prints any exceptions that occur.

Example Language: **PHP** (bad code)

```
try {
  openDbConnection();
}
//print exception message that includes exception message and configuration file location
catch (Exception $e) {
  echo 'Caught exception: ', $e->getMessage(), '\n';
  echo 'Check credentials in config file at: ', $Mysql_config_location, '\n';
}
```

If an exception occurs, the printed message exposes the location of the configuration file the script is using. An attacker can use this information to target the configuration file (perhaps exploiting a Path Traversal weakness). If the file can be read, the attacker could gain credentials for accessing the database. The attacker may also be able to replace the file with a malicious one, causing the application to use an arbitrary database.

In the example below, the method getUserBankAccount retrieves a bank account object from a database using the supplied username and account number to query the database. If an SQLException is raised when querying the database, an error message is created and output to a log file.

Example Language: **Java** (bad code)

```
public BankAccount getUserBankAccount(String username, String accountNumber) {
  BankAccount userAccount = null;
  String query = null;
  try {
    if (isAuthorizedUser(username)) {
      query = "SELECT * FROM accounts WHERE owner = "
      + username + " AND accountID = " + accountNumber;
      DatabaseManager dbManager = new DatabaseManager();
      Connection conn = dbManager.getConnection();
      Statement stmt = conn.createStatement();
      ResultSet queryResult = stmt.executeQuery(query);
      userAccount = (BankAccount)queryResult.getObject(accountNumber);
    }
  } catch (SQLException ex) {
    String logMessage = "Unable to retrieve account information from database,\nquery: " + query;
    Logger.getLogger(BankManager.class.getName()).log(Level.SEVERE, logMessage, ex);
  }
  return userAccount;
}
```

The error message that is created includes information about the database query that may contain sensitive information about the database or query logic. In this case, the error message will expose the table name and column names used in the database. This data could be used to simplify other attacks, such as SQL injection (CWE-89) to directly access the database.

STAGE – 2

OVERVIEW :-

Nessus is one of the most widely used vulnerability scanning tools developed by Tenable. It is designed to help security professionals identify vulnerabilities, misconfigurations, and security weaknesses in systems, networks, and applications. Nessus operates as a remote

vulnerability scanner, meaning it does not need to be installed directly on the target machine. Instead, it scans systems over the network by sending different types of requests and analyzing the responses to detect potential security issues.

From my understanding, Nessus works mainly through a plugin-based architecture. Plugins are small scripts or modules that check for specific vulnerabilities. Each plugin is designed to detect a particular type of weakness such as outdated software, insecure configurations, weak encryption protocols, or exposed services. Nessus has thousands of plugins available, which makes it capable of identifying a wide range of vulnerabilities across operating systems, network devices, databases, and web applications. These plugins are continuously updated to detect newly discovered vulnerabilities.

When a scan is initiated, Nessus first performs host discovery and service identification. During this stage, the scanner sends various network probes using protocols such as TCP, UDP, and ICMP to identify active hosts, open ports, running services, and the operating system of the target machine. This step helps Nessus understand the environment before performing deeper security checks. Once the services are identified, Nessus runs appropriate plugins against those services to test for known vulnerabilities.

Another important aspect I understood about Nessus is its ability to perform automated vulnerability assessments. After scanning the target system, Nessus generates a detailed report containing information about each detected vulnerability. The report includes the vulnerability name, severity level (Critical, High, Medium, Low, or Informational), description, affected port or service, possible impact, and recommended remediation steps. This structured reporting helps administrators and security teams prioritize which vulnerabilities need immediate attention.

Nessus is also very flexible in terms of how scans can be performed. It can scan single hosts, multiple systems in a network, specific ports, or entire subnets. Scans can be executed manually or scheduled to run automatically at regular intervals. This allows organizations to continuously monitor their systems and identify new vulnerabilities as they appear. The tool also provides configuration options that allow users to customize scanning policies depending on the environment and security requirements.

Another benefit of Nessus is its user-friendly interface, which makes it relatively easy to configure scans and analyze results even for users who are new to vulnerability assessment. The reports generated by Nessus are detailed and structured, making them useful for both technical teams and management when evaluating the security posture of an organization.

Overall, from my understanding, Nessus is a powerful and efficient vulnerability scanning tool that helps organizations identify security weaknesses before attackers can exploit them. By regularly scanning systems and addressing the vulnerabilities identified in the reports, organizations can significantly improve their overall cybersecurity posture. Nessus plays an important role in proactive security management and is widely used in penetration testing, vulnerability assessment, and network security auditing.

Target website : http://testfire.net/

Target IP address :- 65.61.137.117

LIST OF VULNERABILITY :

S.NO	VULNERABILITY NAME	SEVERITY	PLUGINS

01.	**TLS Version 1.0 Protocol Detection**	**Medium**	**104743**
02.	**TLS Version 1.1 Deprecated Protocol**	**Medium**	**157288**
03.	**TLS Version 1.1 Protocol Detection**	**Info**	**121010**
04.	**SSL/TLS DiffieHellman Modulus <= 1024 Bits (Logjam)**	**Low**	**83875**
05.	**ICMP Timestamp Request Remote Date Disclosure**	**Low**	**10114**
06.	**Nessus SYN scanner**	**Info**	**11219**
07.	**Additional DNS Hostnames**	**Info**	**46180**
08.	**HyperText Transfer Protocol (HTTP) Information**	**Info**	**24260**
09.	**HSTS Missing From HTTPS Server**	**Info**	**84502**
10.	**Nessus SYN scanner**	**Info**	**11219**

REPORT:-

1.

Vulnerability Name:- TLS Version 1.0 Protocol Detection **severity**

: - Medium

Plugin:- 104743

Port :- 443

Description:- The remote service accepts connections encrypted using TLS 1.0. TLS 1.0 has a number of cryptographic design flaws. Modern implementations of TLS 1.0 mitigate these problems, but newer versions of TLS like 1.2 and 1.3 are designed against these flaws and should be used whenever possible.
As of March 31, 2020, Endpoints that aren't enabled for TLS 1.2 and higher will no longer function properly with major web browsers and major vendors.
PCI DSS v3.2 requires that TLS 1.0 be disabled entirely by June 30, 2018, except for POS POI terminals (and the SSL/TLS termination points to which they connect) that can be verified as not being susceptible to any known exploits.

Solution:- Enable support for TLS 1.2 and 1.3, and disable support for TLS 1.0.

Business Impact::- Using outdated encryption protocols like TLS 1.0 can expose sensitive data to interception and cryptographic attacks such as Man-in-the-Middle (MITM) attacks. Attackers may exploit these weaknesses to decrypt or manipulate secure communications, leading to data breaches, loss of customer trust, and non-compliance with security standards such as PCI DSS.

2.

Vulnerability Name:- TLS Version 1.1 Deprecated Protocol **severity**

: - Medium

Plugin:- 157288

Port :- 443

Description:- The remote service accepts connections encrypted using TLS 1.1. TLS 1.1 lacks support for current and recommended cipher suites. Ciphers that support encryption before MAC computation, and authenticated encryption modes such as GCM cannot be used with TLS 1.1

As of March 31, 2020, Endpoints that are not enabled for TLS 1.2 and higher will no longer function properly with major web browsers and major vendors.

Solution:- Enable support for TLS 1.2 and/or 1.3, and disable support for TLS 1.1.

Business Impact::- Allowing deprecated protocols such as TLS 1.1 increases the risk of attackers exploiting weaknesses in outdated cryptographic mechanisms. This could potentially lead to man-in-the-middle (MITM) attacks, interception of sensitive data, or unauthorized access to confidential information transmitted between the client and the server. Additionally, continued support for deprecated protocols may result in non-compliance with modern security standards and regulatory requirements, and compatibility issues with modern browsers and applications that only support TLS 1.2 or higher.

3.

Vulnerability Name:- TLS Version 1.1 Protocol Detection **severity**

: - Info

Plugin:- 121010

Port :- 443

Description:-The remote service accepts connections encrypted using TLS 1.1.

TLS 1.1 lacks support for current and recommended cipher suites.

Ciphers that support encryption before MAC computation, and authenticated encryption modes such as GCM cannot be used with TLS 1.1

As of March 31, 2020, Endpoints that are not enabled for TLS 1.2 and higher will no longer function properly with major web browsers and major vendors.

Solution:- Enable support for TLS 1.2 and/or 1.3, and disable support for TLS 1.1.

Business Impact::- Allowing the use of deprecated protocols such as TLS 1.1 may expose the system to potential security risks due to weaker cryptographic protection. Attackers may exploit vulnerabilities in outdated encryption methods to intercept or manipulate sensitive data transmitted between the client and server.

Additionally, supporting deprecated protocols may lead to non-compliance with modern security standards and best practices, and it may cause compatibility issues with modern browsers, applications, and services that only support TLS 1.2 or higher.

Vulnerability Name:-

4.

SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)

severity : - Low **Plugin:-**

83875

Port :- 443

Description:- The remote host allows SSL/TLS connections with one or more Diffie-Hellman moduli less than or equal to 1024 bits. Through cryptanalysis, a third party may be able to find the shared secret in a short amount of time (depending on modulus size and attacker resources). This may allow an attacker to recover the plaintext or potentially violate the integrity of connections.

Solution:- Reconfigure the service to use a unique Diffie-Hellman moduli of 2048 bits or greater.

Business Impact::- Using weak Diffie-Hellman parameters may allow attackers to exploit cryptographic weaknesses and potentially decrypt sensitive information transmitted between clients and the server. This could lead to exposure of confidential data, session hijacking, or manipulation of communication integrity.

Additionally, weak encryption configurations may result in non-compliance with modern security standards and best practices, increasing the risk of security incidents and reducing trust in the system's communication security.

5.

Vulnerability Name:- ICMP Timestamp Request Remote Date Disclosure **severity**

: - Low

Plugin:- 10114

Port :- N/A (Network level)

Description:- The remote host answers to an ICMP timestamp request. This allows an attacker to know the date that is set on the targeted machine, which may assist an unauthenticated, remote attacker in defeating time-based authentication protocols.

Timestamps returned from machines running Windows Vista / 7 / 2008 / 2008 R2 are deliberately incorrect, but usually within 1000 seconds of the actual system time.

Solution:- Filter out the ICMP timestamp requests (13), and the outgoing ICMP timestamp replies (14).

Business Impact::- Allowing ICMP timestamp responses may expose the system's internal time information to remote attackers. Although the risk is relatively low, attackers may use this information to assist in reconnaissance activities or exploit time-based authentication mechanisms.

Disclosing system time information may also help attackers analyze system behavior and prepare further targeted attacks, increasing the overall exposure of the network environment.

6.

Vulnerability Name:- Nessus SYN scanner

severity : - Info

Plugin:- 11219

Port :- All ports

Description:- This plugin is a SYN 'half-open' port scanner. It shall be reasonably quick even against a firewalled target.

Note that SYN scans are less intrusive than TCP (full connect) scans against broken services, but they might cause problems for less robust firewalls and also leave unclosed connections on the remote target, if the network is loaded.

Solution:- Protect your target with an IP filter.

Business Impact::- This finding does not represent a direct security vulnerability. It simply indicates that a SYN scan technique was used during the vulnerability assessment. However, if external attackers are able to perform similar scans, they may identify open ports and services

Vulnerability Name:-

running on the system, which could assist them in performing further reconnaissance or targeted attacks.

7.

Additional DNS Hostnames

severity : - Info **Plugin:-**

46180

Port :- 80, 443

Description:- Hostnames different from the current hostname have been collected by miscellaneous plugins. Nessus has generated a list of hostnames that point to the remote host. Note that these are only the alternate hostnames for vhosts discovered on a web server.

Different web servers may be hosted on name-based virtual hosts.

Solution:- If you want to test them, re-scan using the special vhost syntax, such as :

www.example.com[192.0.32.10]

Business Impact::- This finding does not indicate a direct security vulnerability. However, the presence of additional DNS hostnames may indicate multiple applications or services hosted on the same server. If these hostnames are not properly monitored or secured, attackers may identify hidden or less-protected virtual hosts, which could expose additional attack surfaces.

8.

Vulnerability Name:- TLS Version 1.2 Protocol Detection **severity**

: - Info

Plugin:- 136318

Port :- 80, 443

Description:- The remote service accepts connections encrypted using TLS 1.2. Transport Layer Security (TLS) 1.2 is a widely used cryptographic protocol designed to provide secure communication over a network.

TLS 1.2 supports modern cryptographic algorithms and secure cipher suites that help protect data confidentiality and integrity during transmission. The detection of TLS 1.2 indicates that the server supports a secure protocol version for encrypted communication.

This plugin simply reports the presence of TLS 1.2 and does not indicate a security vulnerability.

Solution:- Ensure that TLS 1.2 and/or TLS 1.3 are enabled for secure communications. Additionally, disable older and deprecated protocols such as TLS 1.0 and TLS 1.1 to maintain strong security standards.

Business Impact::- This finding does not represent a security vulnerability. Instead, it confirms that the system supports TLS 1.2, which is considered a secure protocol for encrypted communication. However, organizations should ensure that deprecated protocols such as TLS 1.0 and TLS 1.1 are disabled to maintain compliance with modern security best practices.

9.

Vulnerability Name:- HTTP Server Type and Version **severity**

: - Low

Plugin:- 10107

Port :- 443

Description:- This plugin attempts to determine the type and the version of the remote web server.

Solution:- Configure the web server to hide or suppress the server version information in HTTP response headers. This can typically be done by modifying the web server configuration to remove or mask the Server header.

For example:

- Disable or modify server tokens in the web server configuration.
- Use security modules or reverse proxies to remove version disclosure.

Business Impact::- Exposing the web server type and version information may assist attackers in identifying specific vulnerabilities associated with the disclosed server software. This information can be used during reconnaissance to plan targeted attacks against known weaknesses in that server version.

Although this issue alone does not directly compromise the system, it increases the attack surface by revealing internal system details that could be leveraged in further attacks.

Vulnerability Name:-

10.

HyperText Transfer Protocol (HTTP) Information

severity : - Info

Plugin:- 24260

Port :- 80, 443

Description:- This test gives some information about the remote HTTP protocol - the version used, whether HTTP Keep-Alive is enabled, etc...
This test is informational only and does not denote any security problem.

Solution:- No immediate action is required since this is an informational finding. However, administrators should review HTTP configurations regularly and ensure that the web server follows secure configuration best practices.

Business Impact::- This finding does not represent a direct security risk. However, the information disclosed about the HTTP service configuration may assist attackers during reconnaissance activities, helping them understand the server environment and potentially identify other vulnerabilities associated with the HTTP service.

STAGE – 3

1. SOC - How Zscaler Enhances Security Operations Center

From my project, I learned that a Security Operations Center (SOC) is the team responsible for monitoring and responding to security threats. In the legacy VPN architecture, the SOC faced many challenges because users got direct access to the internal network. Attackers could move laterally once inside, making it hard for the SOC to track them. With Zscaler Zero Trust, the SOC's job becomes more focused. Zscaler works on "never trust, always verify" and connects users only to applications, not to the network. This means the SOC doesn't have to worry about lateral movement anymore. They can monitor everything through the Zscaler cloud console and focus on user identities and application access. The SOC also gets detailed logs of every connection, which helps them investigate incidents faster. As mentioned in my project, Zscaler transforms how the SOC operates by making the network invisible to users and attackers alike.

2. How Zscaler Improves the Containment Phase of SOC Cycle

The SOC cycle includes monitoring, detection, analysis, containment, eradication, and recovery. In my project, I understood how Zscaler improves the containment phase dramatically. In the old VPN model, when the SOC detected a compromised account, they had to manually disable the VPN account and update firewall rules. This took hours, and during that time attackers could move laterally and cause more damage. But with Zscaler, containment happens in seconds. The SOC analyst can simply go to the Zscaler cloud dashboard and deactivate the user or block the device. All active sessions are terminated immediately across the globe. Because the user was never placed on the internal network, there is no lateral movement to worry about. This makes the entire SOC cycle faster and more effective, reducing the impact of any security incident.

3. SIEM - Integrating Zscaler Logs with Security Information and Event Management

Vulnerability Name:-

SIEM is a tool that collects and correlates logs from different sources to detect security threats. My project explains how Zscaler integrates with SIEM systems like Splunk or Sentinel. Zscaler sends detailed logs to the SIEM containing information about each connection - which user accessed which application, from where, what device they used, how much data was

transferred, and the device posture. When these logs go into the SIEM, they can be correlated with other data like authentication logs from Azure AD and vulnerability scan results from Nessus. This gives the SOC a complete picture of what's happening across the organization. For example, if a user accesses an application from an unusual location and downloads large amounts of data, the SIEM can flag this as potential data theft and create an alert for investigation.

4. How Zscaler Logs Enhance SIEM Correlation

The SIEM cycle involves collecting data, normalizing it, correlating events, analyzing patterns, and generating alerts. With Zscaler integration, each step of this cycle becomes more powerful. First, Zscaler collects data from every user connection across the globe and sends it to the SIEM. The SIEM normalizes this data so it can be compared with logs from other sources. Then it correlates events - for example, matching a Zscaler log showing access to a suspicious website with threat intelligence from MISP. The analysis phase uses rules and machine learning to identify patterns that indicate attacks. Finally, alerts are generated for the SOC team to investigate. The detailed, identity-rich data from Zscaler makes every step of the SIEM cycle more accurate and helps reduce false positives.

5. MISP - Threat Intelligence Integration with Zscaler

MISP (Malware Information Sharing Platform) is an open-source threat intelligence platform that shares Indicators of Compromise (IOCs) within the security community. My project explains MISP integration in detail. Our security team configures MISP to push threat intelligence feeds to the Zscaler cloud via API. These feeds contain malicious domains, IP addresses of command-and-control servers, URLs of phishing sites, and file hashes of known malware. When a user tries to access anything that matches these IOCs, Zscaler Internet Access (ZIA) automatically blocks the request in real-time. For example, if a new ransomware campaign is discovered and added to MISP, within minutes Zscaler will start blocking those ransomware sites for all our users. This turns our security from reactive to proactive and stops threats before they reach our users.

6. My College Network Information - Current VPN Problems

In my college, we currently use a legacy VPN for remote access. This VPN operates on the old "castle-and-moat" model. The VPN concentrator is exposed to the public internet with open ports, making it visible to attackers who constantly scan for vulnerabilities. Once a user authenticates, they get an IP address on the internal network and can access many systems. This creates two major problems. First, the attack surface is large because the VPN is visible to anyone on the internet. Second, if an attacker compromises a student or faculty account, they can move laterally across the network to find valuable targets like the Student Information System (SIS), research data repositories, administrative file servers, and ERP systems. This is exactly why my project aims to replace the legacy VPN with Zscaler Zero Trust.

7. How I Would Deploy SOC in My College - A Zscaler-Based Approach

Based on the deployment methodology in my project, here is how I would deploy a SOC in my college. First, we would deploy Zscaler App Connectors as lightweight VMs in our data center. These connectors only need outbound internet access to connect to the Zscaler cloud - no inbound firewall ports need to be opened, which eliminates the attack surface. Then we would install the Zscaler Client Connector on all student, faculty, and staff devices. We would integrate Zscaler with our identity provider (like Azure AD) so users authenticate with their college credentials. Next, we would configure Zscaler to forward all logs to our SIEM for centralized monitoring. The SOC team would be trained to use the Zscaler Admin Portal for live monitoring and the SIEM for deeper investigation. Finally, we would follow the phased rollout - pilot group first, then faculty, then staff, then students - before decommissioning the legacy VPN completely.

8. Threat Intelligence - Making Zscaler Smarter with Context

Threat intelligence is information about threats that helps us make better security decisions. In my project, threat intelligence comes from multiple sources - Zscaler cloud logs showing real-time activity, external threat feeds from MISP, identity provider logs from Azure AD showing authentication patterns, and endpoint detection tools showing device health. This intelligence enables adaptive access control. For example, if a faculty member tries to access the finance application from an IP address that threat intelligence flags as high-risk (like a

known TOR exit node), Zscaler can automatically require step-up authentication or block the session entirely. If threat intelligence identifies a user's source IP as associated with malicious activity, Zscaler can restrict access to read-only mode. This means access decisions are based on risk context, not just on whether someone has a valid password.

9. Incident Response - How Zscaler Transforms Response Workflow

My project explains incident response in detail. In the Zscaler Zero Trust model, incident response is fundamentally improved. When the SOC detects a compromised account - for example, a staff member downloading 50GB of research data at 3 AM from an unusual country - they can act immediately. The analyst opens the Zscaler cloud dashboard, finds the user profile, and clicks "Deactivate User" or "Block Device." This action terminates all active Zscaler connections instantly and prevents any new authentication attempts. Because the user was never on the network, there is no lateral movement to contain. The compromised device can be wiped remotely, the user's password can be reset, and once everything is clean, access can be restored. All subsequent activity is closely monitored. The detailed logs from Zscaler provide a complete audit trail for forensic analysis, and the whole process takes minutes instead of hours.

10. QRadar - Understanding the SIEM Tool

IBM QRadar is mentioned in my project conclusion as a SIEM tool. QRadar collects and analyzes log data from across an organization to provide centralized visibility. In the context of my project, QRadar would receive logs from Zscaler, Nessus vulnerability scans, Azure AD authentication, and other sources. What makes QRadar powerful is its correlation engine. It can connect events that might seem unrelated. For example, it could correlate a failed login attempt in Azure AD with a successful Zscaler connection from a different country 5 minutes later, and flag this as possible account takeover. QRadar also has built-in threat intelligence and machine learning to detect anomalies. It gives the SOC team a central dashboard to monitor everything, investigate incidents, and generate reports for compliance. For a college like ours, QRadar would help the SOC team respond to threats faster and more effectively.

CONCLUSION:

Stage 1: What I understood from Web Application Testing

In Stage 1, I learned that web application testing is critical for finding security vulnerabilities before attackers can exploit them. Using the OWASP Top 10 helps us understand common vulnerabilities like improper input validation, cross-site scripting, and broken authentication. I understood that testing should happen throughout development, not just at the end. Both automated tools and manual testing are important because automated tools can miss complex logic flaws. The practice website and main website vulnerability tables in my project showed me real examples of vulnerabilities like missing security headers, weak cipher suites, and information disclosure issues.

Stage 2: What I understood from the Nessus Report

Stage 2 taught me how Nessus works as a vulnerability scanner. It sends probes to target systems and identifies weaknesses like TLS 1.0 protocol detection, missing HSTS headers, and weak Diffie-Hellman moduli. The report gives each vulnerability a severity score (MEDIUM, LOW, INFO) and tells us how to fix it. I learned that we need to prioritize fixing vulnerabilities based on their severity and business impact. The Nessus report also showed me that some findings are informational and don't need immediate action but are still useful for understanding the environment. Regular scanning is important because new vulnerabilities are discovered every day.

Stage 3: What I understood from SOC/SIEM and Zscaler Zero Trust

From Stage 3, I understood how SOC, SIEM, and Zero Trust work together to protect an organization. The SOC is the team that monitors and responds to threats, and SIEM tools help them by correlating data from multiple sources. What made this project special is seeing how Zscaler's Zero Trust architecture makes these concepts work better. By replacing the legacy VPN, we eliminate the attack surface completely because no inbound ports are exposed. We stop lateral movement because users are never placed on the network - they only connect

to specific applications. Incident response goes from taking hours to taking seconds because containment is done instantly from the cloud. Integrating MISP for threat intelligence makes security proactive. Overall, I learned that modern security is about verifying everything, trusting nothing, and protecting users wherever they work.

FUTURE SCOPE:

Stage 1: Future Scope of Web Application Testing

The future of web application testing will see more use of AI and machine learning to automate test case generation and find complex vulnerabilities. Testing will shift left, meaning it will happen earlier in the development process. Cloud-based testing platforms will become more common, offering scalability and flexibility. As applications become more complex with APIs and microservices, testing tools will need to evolve to secure these interconnected systems. There will be increased focus on user experience testing along with security testing.

Stage 2: Future Scope of the Testing Process

The testing process will become more continuous and automated. AI will help prioritize which vulnerabilities to fix first by predicting which ones are most likely to be exploited. Vulnerability scanners will integrate more closely with other security tools, enabling automated workflows. The goal is to move from point-in-time assessments to real-time visibility of an organization's security posture. Cloud-based testing platforms will offer advantages like scalability, flexibility, and cost-effectiveness. Testing automation will help save time and resources while improving quality.

Stage 3: Future Scope of SOC/SIEM

As mentioned in my project, the future of SOC and SIEM includes several exciting developments. We will see AI-driven threat hunting that can detect anomalies automatically by baselining normal user behavior. Integration with SOAR (Security Orchestration, Automation and Response) platforms will enable fully automated incident response playbooks. The Zero Trust journey will expand to protect workloads, branch offices, and IoT devices. Advanced

Data Loss Prevention (DLP) will help protect sensitive research data and student information. Machine learning will help identify insider threats and compromised accounts by analyzing patterns of application access, data transfer volumes, and login times. We can expect more intelligent and adaptive systems that can assess risk in real-time and respond automatically.

Topics Explored:

SOC (Security Operations Center), SOC Cycle (Incident Response), SIEM (Security Information and Event Management), SIEM Cycle, MISP (Malware Information Sharing Platform), Threat Intelligence, Incident Response, Zero Trust Architecture, Zscaler Private Access (ZPA), Zscaler Internet Access (ZIA), Lateral Movement, Attack Surface Reduction, Identity Provider Integration (Azure AD), Application Segmentation, QRadar, Legacy VPN Limitations.

Tools Explored:

Zscaler (ZPA and ZIA), IBM QRadar, MISP, Nessus, Nmap, Metasploit, Kali Linux, OWASP ZAP, Burp Suite, Azure AD, Splunk, Sentinel.